Exercises · AI in Java

AI08 Exercises

Work each one before reading its solution. Exercises 1 and 4 include arithmetic. Practise Exercise 6 out loud.

Exercise 1: Size the timeouts

A controller calls `ClaudeModel` with a `timeout` of 45 seconds and `maxRetries` of 3. The API gateway times out at 60 seconds, and the mobile client at 30.

(a) What's the worst-case time for the call? **(b)** Which timeouts will fire first in a slow overload? **(c)** Propose settings for a 25-second request. **(d)** What would you change for answers that need 90 seconds?

Solution

(a) 45 × 4 = **180 seconds**, plus backoff.

(b) The client at 30 seconds, then the gateway at 60, while the service keeps working, and keeps paying.

(c) For example, a timeout of 11 seconds and 1 retry, so 22 seconds plus backoff. Check this against real latency, since a low timeout can fail ordinary requests.

(d) Stream, so early bytes keep the connection alive and the user sees progress. Or move it to a queue, and notify on completion.

Exercise 2: Review a controller

```java
@PostMapping("/chat")
String chat(@RequestParam String message, @RequestParam String customerId) {
    AnthropicClient client = AnthropicOkHttpClient.builder().apiKey(KEY).build();
    try {
        return client.messages().create(MessageCreateParams.builder()
                .model("claude-opus-5-5").maxTokens(1000L).temperature(0.2)
                .addUserMessage("Customer " + customerId + ": " + message).build())
                .content().get(0).text().get().text();
    } catch (Exception e) {
        return "Error: " + e.getMessage();
    }
}
```

List at least seven problems.

Solution

1. A new client per request, never closed, so connections leak.
2. `KEY` looks like a constant in code.
3. `customerId` comes from the request, so anyone can claim to be anyone.
4. `temperature` returns a 400 on Opus 5.5.
5. `content().get(0).text().get()` assumes the first block is text. On Opus 5.5, a thinking block can come first, so this throws.
6. `stopReason` is ignored, so a truncated answer looks complete. `max_tokens` of 1,000 makes that likely.
7. A broad catch hides retryable versus permanent, and the message may leak details to the caller.
8. No validation of `message`, such as its length.
9. Customer ID is placed in the user message, where it isn't needed.

Exercise 3: Design the ledger states

For `process_refund`, draw the states of a ledger row, and say what to do when a retry finds each one.

Solution

State	Meaning	A retry should
`STARTED`	Claimed. Outcome unknown	**Not run.** Report "in progress". Reconcile with billing using the key
`OK`	Refund confirmed	Return the stored result
`DENIED`	Policy refused	Return the same denial. A new attempt needs a new call ID
`FAILED`	Tool reported a failure	Return it. If retryable, the model may call again with a new ID
`CRASHED`	Threw unexpectedly	Treat like `STARTED` for money, since it may have reached billing

Note the last row: this module's code returns a generic transient error after a crash. For side-effecting tools, a retry with a new ID could double-pay, unless billing has its own idempotency, using a business key such as order plus amount.

Exercise 4: Choose and cost

Each needs a shape (sync, Kafka, Batch API or workflow), and a monthly cost, on Sonnet 5.5 ($2 / $10), at 3,000 input and 300 output tokens each.

(a) 500,000 document summaries overnight. **(b)** A chat assistant with 300,000 messages. **(c)** 100,000 new-ticket classifications, within two minutes. **(d)** Why might (c)'s cost be far lower in practice?

Solution

Per call: $0.006 + $0.003 = **$0.009.**

(a) Batch API, at half price: 500,000 × $0.0045 = **$2,250.**

(b) Sync with streaming: 300,000 × $0.009 = **$2,700.**

(c) Kafka workers: 100,000 × $0.009 = **$900.**

(d) A classifier doesn't need 3,000 tokens of input or 300 of output, and Haiku 5.5 might do if evals agree. Most of the prompt can be cached too.

Exercise 5: Write the tests

The banking assistant has a rule: "An answer must never state a total not returned by a tool." Write test ideas at three layers, and say what each proves.

Solution

1. **Scripted model, unit.** Script a reply stating "$99", with a tool result of "41.30". Assert that the checker flags it. This proves that our checker works, and says nothing about how often the model does this.
2. **Fake API or recorded replies.** Feed real recorded replies through the checker, to confirm that it handles real formatting, such as commas and currency symbols.
3. **Evals, real model.** Run real questions, and measure how often the rule is broken, by segment, and how often the checker raises false alarms. This is the only one measuring the model.

Also keep `figure` events from code, so that the display doesn't depend on the text at all.

Exercise 6: Defend a decision

A product manager says: "The assistant could do so much more if it could move money. Competitors will. Add a `transfer_funds` tool, and we'll put strong rules in the prompt."

Respond in about a minute.

Model answer

"I'd like customers to be able to act from the assistant, and I don't think a prompt can carry this risk. Prompts lower failure rates, but they don't remove them. Transaction descriptions are text that outsiders control, so a successful injection with a `transfer_funds` tool has a direct path to moving money. On a read-only assistant, the worst outcome is a misleading sentence.

My proposal gets most of the value. The assistant drafts the transfer, and the customer confirms it in the existing payment flow, with its usual authentication and limits. The model proposes, and the established system decides and executes. Customers get the convenience, with no new way to move money.

Then I'd measure: how often people use drafts, and where they drop off. If there's a strong case for going further, we'd take a specific design to risk and compliance.

I'd reconsider if we had strong evidence that injection was contained for this tool, plus compliance approval, and limits such as saved payees only. Even then, I'd want the money rules in the payment system, not in the assistant."